

Speicherallokation im Kernel

Kontext, Kontiguität und robuste Freigabepfade

3.07

Vier Fragen führen zur passenden Speicher-API

01 Kontext

Darf der aktuelle Ausführungskontext schlafen?

02 Größe

Wie viele Bytes oder Pages werden benötigt?

03 Kontiguität

Muss der Speicher physisch zusammenhängend sein?

04 Lebensdauer

Wer besitzt und wer gibt die Allokation frei?

Ziel Allokation und Freigabe als ein gemeinsames Designproblem behandeln.

Mehrere Allocator-Schichten bedienen verschiedene Größen

Objekte

SLUB / kmem_cache

Bytebereiche

kmalloc-Familie

Virtuelle Bereiche

vmalloc / kvmalloc

Pages

Page Allocator

Physische Blöcke

Buddy-System

Höhere Schichten nutzen darunterliegende Mechanismen und ergänzen Caches oder Metadaten.

Der Ausführungskontext entscheidet über zulässiges Warten

Schlafen erlaubt

`GFP_KERNEL`

Normalfall im Prozesskontext.
Reclaim und I/O können möglich sein.

Schlafen verboten

`GFP_ATOMIC`

Nur wenn wirklich erforderlich.
Kann Reserven nutzen und leichter scheitern.

Faustregel: Zuerst den Kontext bestimmen – danach den GFP-Maskentyp wählen.

Die API folgt Größe und Kontiguitätsanforderung

`kmalloc()`

klein bis moderat · physisch zusammenhängend

`kfree()``vmalloc()`

groß · nur virtuell zusammenhängend

`vfree()``kvmalloc()`

kmalloc versuchen, sonst vmalloc

`kvfree()``alloc_pages()`

ganze, physisch zusammenhängende Pages

`__free_pages()``kmem_cache_alloc()`

viele gleichartige Objekte

`kmem_cache_free()`

Die kmalloc-Familie deckt den häufigsten Fall ab

```
ptr = kmalloc(size, GFP_KERNEL);
```

```
ptr = kzalloc(size, GFP_KERNEL);
```

A**kmalloc**

Inhalt ist nach erfolgreicher Allokation nicht automatisch initialisiert.

B**kzalloc**

Zusätzlich mit Null initialisieren.

C**kfree**

NULL ist zulässig; Pointer danach nicht erneut verwenden.

Jede Allokation kann fehlschlagen: Rückgabewert vor der Nutzung auf NULL prüfen.

Größenberechnung ist Teil der Speichersicherheit

Riskant

```
kmalloc(n * sizeof(*items), GFP_KERNEL)
```

Bevorzugt

```
kmalloc_array(n, sizeof(*items), GFP_KERNEL)
```

```
kcalloc(n, sizeof(*items), GFP_KERNEL)
```

mit Nullinitialisierung

Warum?

Array-Helfer erkennen Überläufe in der Multiplikation der Größenparameter.

Für Strukturen mit flexiblem Array passende size-/struct-Helfer verwenden.

vmalloc löst virtuelle und physische Kontiguität

Virtueller Bereich

zusammenhängende virtuelle Adressen

Physische Pages

PFN 14

PFN 88

PFN 31

PFN 205

PFN 7

PFN 121

PFN 54

Vorteil: große virtuelle Bereiche ohne physisch zusammenhängenden Block.

Kosten: zusätzliche Page Table und andere Zugriffseigenschaften als kmalloc-Speicher.

kvmalloc kombiniert Bequemlichkeit mit einem Fallback

```
ptr = kvmalloc(size, GFP_KERNEL);
```

```
kvfree(ptr);
```

- 1 `kmalloc` versuchen direkter, physisch zusammenhängender Bereich
- 2 bei Bedarf `vmalloc` virtuell zusammenhängender Fallback
- 3 mit `kvfree` freigeben unabhängig vom erfolgreichen Pfad

Nur verwenden, wenn `vmalloc`-Speicher für den Verbraucher zulässig ist.

Der Page Allocator arbeitet in Zweierpotenzen von Pages

```
page = alloc_pages(GFP_KERNEL, order);
```

```
__free_pages(page, order);
```

order 0

1 Page

order 1

2 Pages

order 2

4 Pages

order 3

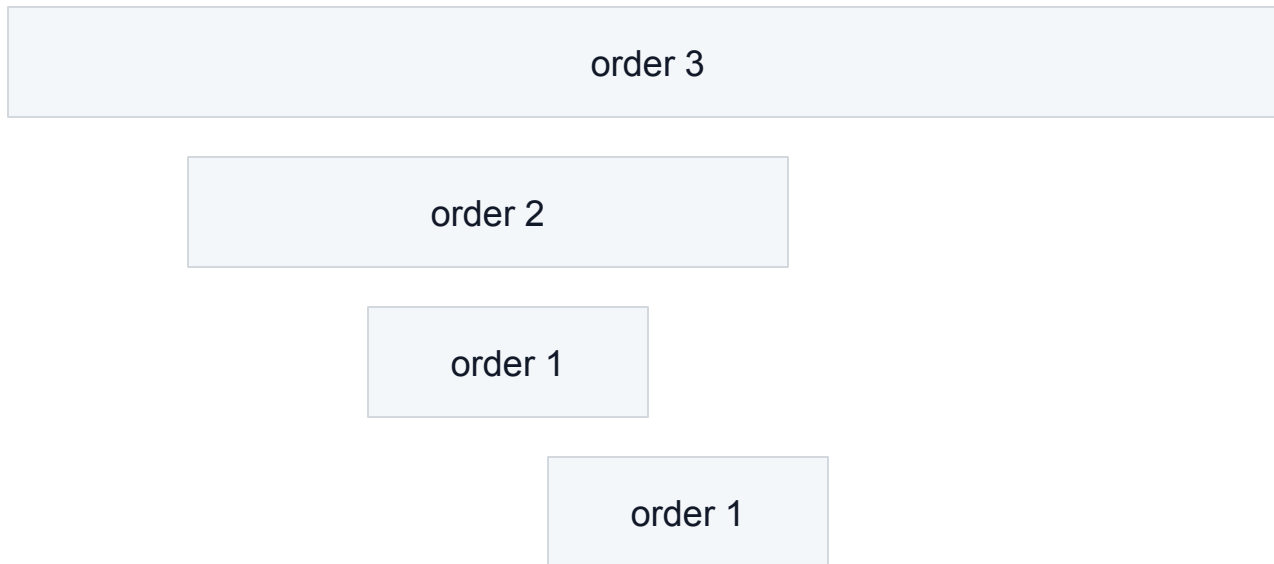
8 Pages

Ein höherer order verlangt einen größeren physisch zusammenhängenden Block.

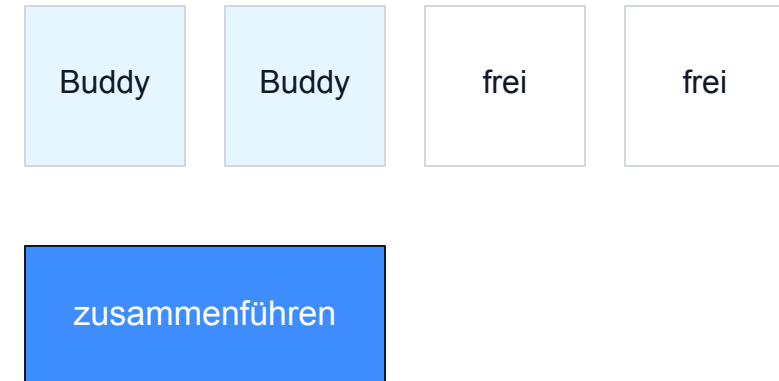
Unter Fragmentierung können große Orders trotz freien Gesamtspeichers scheitern.

Das Buddy-System teilt und vereinigt freie Blöcke

Allokation

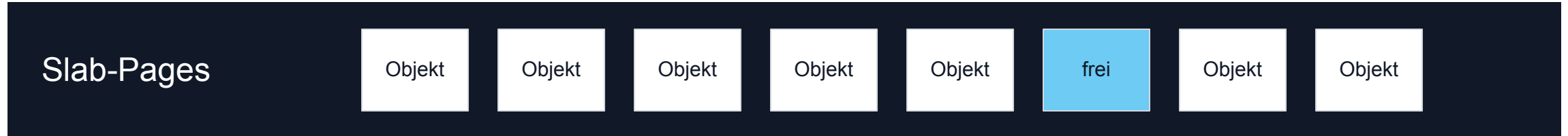


Freigabe



Teilen liefert die kleinste ausreichende Ordnung; Freigeben erlaubt das rekursive Zusammenführen passender Buddies.

SLUB recycelt Speicher für häufige Objektgrößen



01

Caches

Objekte gleicher Größe und Konstruktion bündeln.

02

Wiederverwendung

Freie Slots ohne neue Page-Allokation nutzen.

03

Debugging

Redzones, Poisoning und Tracking optional aktivieren.

Ziel: weniger Verwaltungsaufwand und Fragmentierung bei vielen kleinen Objekten.

Eigene Caches lohnen sich bei vielen gleichartigen Objekten

```
cache = kmem_cache_create(...);
```

```
obj = kmem_cache_alloc(cache, GFP_KERNEL);
```

```
kmem_cache_free(cache, obj);
```

```
kmem_cache_destroy(cache);
```

A Voraussetzung

Feste Objektgröße und klarer Lebenszyklus.

B Nutzen

Caching und optionale Konstruktorlogik.

C Pflicht

Vor dem Destroy müssen alle Objekte zurückgegeben sein.

Für wenige Objekte bleibt kmalloc meist die einfachere Wahl.

Fehlerpfade müssen jede Teilallokation zurückrollen

```
buf = kzalloc(size, GFP_KERNEL);
```

```
if (!buf) return -ENOMEM;
```

```
table = kmalloc_array(n, sizeof(*table),  
GFP_KERNEL);
```

```
if (!table) goto err_buf;
```

Rückrollpfad

```
err_buf:  
    kfree(buf);  
    return -ENOMEM;
```

Regel

In umgekehrter Reihenfolge freigeben – genau einmal, auch bei Teilerfolg.

Vier Regeln machen Allokationen robust

01 Kontext zuerst

Schlafmöglichkeit bestimmt die GFP-Maske.

02 Eigenschaft benennen

Physische oder nur virtuelle Kontiguität klären.

03 Fehler erwarten

NULL prüfen und Größenüberläufe vermeiden.

04 Freigabe koppeln

Passende API und eindeutigen Owner festlegen.

Nächster Schritt: Kernelmodul um dynamischen Speicher und Fehlerpfade erweitern.